ASSIGNMENT 3.2

Self-Interested Utility Optimization

Student	[Student Name]	Roll No	[Roll No]
Course	21CSE525T - Agent Technology	Project	Placement Preparation Agent

Unit 3 - Choosing one training package from 10 options under hard constraints. File: assignments/a3_2/utility_optimizer.py

1. Objective

Choose the best training option for the student by maximising an explicit utility function, subject to hard constraints, and prove that the choice is optimal.

2. Constraints and preferences

- Budget: Rs 12,000. Available hours: 70. Minimum mentor rating: 7.0.
- Weights: skill 0.25, placement boost 0.30, mentor rating 0.15, cost 0.20, time 0.10 (sum = 1).
- The student already knows Arrays, Strings and OOPS. Options that repeat these topics receive a penalty of 0.25 x (overlap / topics).

3. Method

Step 1 - Feasibility filter: any option breaking cost, hours or rating is removed. 4 of the 10 options are rejected (FAANG Interview Track, Full Stack Placement, Free Community Track, Premium 1-on-1 Mentor).

Step 2 - Utility: each feasible option is scored with U(x) using normalised attributes minus the redundancy penalty.

Step 3 - Argmax with proof: exhaustive search over all feasible options. The code prints U(x*) against the runner-up, and states that x* beats every feasible option.

Step 4 - Bundles: combinations of one to three options within budget and hours are checked for the highest total utility.

Step 5 - Boundary tests: zero budget, zero hours, impossible rating and negative budget are each run to confirm the program handles them without crashing.

4. Implementation

```
class SelfInterestedAgent:
    def __init__(self):
        self.budget = 12000            # Rs.
        self.available_hours = 70      # hours before placement season
        self.min_rating = 7.0
        self.mastered = {"Arrays", "Strings", "OOPS"}   # already strong -&gt; penalty
        # w_skill, w_place, w_rating, w_cost, w_time  (must sum to 1.0)
        self.w = (0.25, 0.30, 0.15, 0.20, 0.10)
        assert abs(sum(self.w) - 1.0) &lt; 1e-9, "weights must sum to 1"

    # ---------------- FEASIBILITY (hard constraints) ------------------
    def is_feasible(self, o):
        _id, name, cost, hours, sg, pb, rating, topics = o
        violations = []
        if cost &gt; self.budget:
            violations.append(f"cost {cost}&gt;{self.budget}")
        if hours &gt; self.available_hours:
            violations.append(f"hours {hours}&gt;{self.available_hours}")
        if rating &lt; self.min_rating:
            violations.append(f"rating {rating}&lt;{self.min_rating}")
        return (len(violations) == 0), violations

    # ---------------- UTILITY FUNCTION --------------------------------
    def utility(self, o, max_sg=None):
        _id, name, cost, hours, sg, pb, rating, topics = o
        max_sg = max_sg or max(x[4] for x in OPTIONS)
```

```
        w1, w2, w3, w4, w5 = self.w

        # ---- constraint boundary resilience: guard divide-by-zero ----
        budget = self.budget if self.budget &gt; 0 else 1
        hrs_cap = self.available_hours if self.available_hours &gt; 0 else 1

        n_skill = sg / max_sg
        n_place = pb / 100
        n_rating = rating / 10
        n_cost = max(0.0, 1 - cost / budget)
        n_time = max(0.0, 1 - hours / hrs_cap)

        overlap = len(topics &amp;amp; self.mastered)
        penalty = 0.25 * (overlap / max(len(topics), 1))

        u = (w1 * n_skill + w2 * n_place + w3 * n_rating
             + w4 * n_cost + w5 * n_time) - penalty
        return round(u, 4), {
            "n_skill": round(n_skill, 3), "n_place": round(n_place, 3),
            "n_rating": round(n_rating, 3), "n_cost": round(n_cost, 3),
            "n_time": round(n_time, 3), "penalty": round(penalty, 3),
        }

    # ---------------- OPTIMIZATION SEARCH ----------------------------
    def exhaustive_search(self):
        """Guaranteed-optimal brute force over the feasible set."""
        results, rejected = [], []
        max_sg = max(x[4] for x in OPTIONS)
        for o in OPTIONS:
            ok, why = self.is_feasible(o)
            if not ok:
                rejected.append((o, why))
                continue
            u, parts = self.utility(o, max_sg)
            results.append((u, o, parts))
        results.sort(key=lambda x: -x[0])
        return results, rejected

    def greedy_bundle(self, feasible):
        """Constrained multi-select: maximize total utility within budget+time."""
        best, best_u = None, -1
        pool = [(u, o) for u, o, _ in feasible]
        for r in (1, 2, 3):
            for combo in combinations(pool, r):
                cost = sum(o[2] for _, o in combo)
                hours = sum(o[3] for _, o in combo)
                if cost &lt;= self.budget and hours &lt;= self.available_hours:
                    total = sum(u for u, _ in combo)
                    if total &gt; best_u:
                        best_u, best = total, combo
        return best, round(best_u, 4)
```

5. Result

Six options are feasible. The optimum is **Mock Interview Bundle** with U = 0.6339, against a runner-up (DSA Bootcamp Pro) at 0.5769. The bundle costs Rs 9,700 of Rs 12,000 and uses 20 of 70 hours. The best multi-select bundle has total utility 1.6428 and uses all 70 hours.

Execution output (log file)

Captured from **logs/3.2_utility_optimizer.log** (python3 utility_optimizer.py). Timestamps and some values differ between runs; this is the recorded run.

```
# Activity 3.2_utility_optimizer
# command: python3 utility_optimizer.py
# exit code: 0
# lines: 68

================================================================================================
 SELF-INTERESTED UTILITY OPTIMIZATION :: PLACEMENT PREP AGENT (Assignment 3.2)
================================================================================================
 CONSTRAINTS  budget&lt;=Rs.12,000 | hours&lt;=70 | mentor_rating&gt;=7.0
 WEIGHTS      skill=0.25 place=0.3 rating=0.15 cost=0.2 time=0.1   (sum=1.0)
 MASTERED     ['Arrays', 'OOPS', 'Strings']  -&gt; redundancy penalty applies

------------------------------------------------------------------------------------------------
 SEARCH SPACE : 10 OPTIONS
```

```
-----------------------------------------------------------------------------------------------
 ID Option                          Cost   Hrs  Skill  Boost  Rate
 1  DSA Bootcamp Pro             8,500    60     82     74   8.6
 2  Aptitude Crash Course        3,200    25     45     38   7.4
 3  FAANG Interview Track       14,500    90     95     92   9.4
 4  Core CS Revision Kit         4,100    35     58     51   7.9
 5  Mock Interview Bundle        6,800    20     40     78   9.1
 6  Full Stack Placement        11,200   110     88     69   8.2
 7  Free Community Track             0    80     35     22   5.2
 8  Resume + LinkedIn Lab        2,400    10     18     44   8.0
 9  Product Company Sprint       9,900    55     79     85   8.8
 10 Premium 1-on-1 Mentor       19,800    45     90     88   9.7

-----------------------------------------------------------------------------------------------
 STEP 1 :: CONSTRAINT FILTERING (infeasible options pruned)
-----------------------------------------------------------------------------------------------
   [REJECT] id=3 FAANG Interview Track      violations: cost 14500&gt;12000, hours 90&gt;70
   [REJECT] id=6 Full Stack Placement       violations: hours 110&gt;70
   [REJECT] id=7 Free Community Track       violations: hours 80&gt;70, rating 5.2&lt;7.0
   [REJECT] id=10 Premium 1-on-1 Mentor      violations: cost 19800&gt;12000
   -&gt; 6 feasible / 10 total

-----------------------------------------------------------------------------------------------
 STEP 2 :: UTILITY EVALUATION OF FEASIBLE SET (ranked)
-----------------------------------------------------------------------------------------------
 Rank  ID  Option                        U(x)   components
 1     5   Mock Interview Bundle       0.6339   skill=0.421 place=0.78 rate=0.91 cost=0.433 time=0.714 pen=0.0
 2     1   DSA Bootcamp Pro            0.5769   skill=0.863 place=0.74 rate=0.86 cost=0.292 time=0.143 pen=0.062
 3     9   Product Company Sprint      0.5680   skill=0.832 place=0.85 rate=0.88 cost=0.175 time=0.214 pen=0.083
 4     2   Aptitude Crash Course       0.5544   skill=0.474 place=0.38 rate=0.74 cost=0.733 time=0.643 pen=0.0
 5     8   Resume + LinkedIn Lab       0.5451   skill=0.189 place=0.44 rate=0.8 cost=0.8 time=0.857 pen=0.0
 6     4   Core CS Revision Kit        0.5433   skill=0.611 place=0.51 rate=0.79 cost=0.658 time=0.5 pen=0.062

===============================================================================================
 STEP 3 :: OPTIMAL SINGLE DECISION (argmax U)
===============================================================================================
   SELECTED : [5] Mock Interview Bundle
   U(x*)    = 0.6339
   Cost     = Rs.6,800 / Rs.12,000   (5,200 unused)
   Hours    = 20 / 70
   PROOF    : U(x*) = 0.6339 &gt; U(runner-up 'DSA Bootcamp Pro') = 0.5769
              and U(x*) &gt;= U(x) for ALL 6 feasible x. OPTIMAL.

===============================================================================================
 STEP 4 :: CONSTRAINED MULTI-SELECT (knapsack extension)
===============================================================================================
   + [2] Aptitude Crash Course      U=0.5544  Rs.3,200  25h
   + [8] Resume + LinkedIn Lab      U=0.5451  Rs.2,400  10h
   + [4] Core CS Revision Kit       U=0.5433  Rs.4,100  35h
   TOTAL UTILITY = 1.6428 | cost Rs.9,700/12,000 | hours 70/70

===============================================================================================
 STEP 5 :: ERROR / CONSTRAINT BOUNDARY RESILIENCE TESTS
===============================================================================================
   [Zero budget       ] handled: 0 feasible options -&gt; agent returns NO_VIABLE_PLAN (no crash)
   [Zero hours        ] handled: 0 feasible options -&gt; agent returns NO_VIABLE_PLAN (no crash)
   [Impossible rating ] handled: 0 feasible options -&gt; agent returns NO_VIABLE_PLAN (no crash)
   [Negative budget   ] handled: 0 feasible options -&gt; agent returns NO_VIABLE_PLAN (no crash)
===============================================================================================
```

6. Honest notes for the viva

- The boundary tests print the message NO_VIABLE_PLAN. The function does not return that value; the message is printed by main(). Say so if asked.
- greedy_bundle is in fact an exhaustive search over combinations of up to three options, not a greedy algorithm.
- Weights are chosen by the designer and are not learned from the student.

7. Conclusion

An explicit utility function plus hard constraints gives a choice that is both feasible and provably best among the feasible options.